Problem Statement: Find All Duplicates in an Array

- Description: Given an array of integers, 1 ≤ a[i] ≤ n (n = size of array), some elements appear twice and others appear once. Find all the elements that appear twice in this array.
- Example:
  - Input: [4, 3, 2, 7, 8, 2, 3, 1]
  - Output: [2, 3]

Problem Statement:

Given an array of integers, where `1 ≤ a[i] ≤ n` (n = size of array), some elements appear twice and others appear once. Find all the elements that appear twice in this array.

Example:

Input: `[4, 3, 2, 7, 8, 2, 3, 1]`
Output: `[2, 3]`

Step-by-Step Solution:

Step 1: Initialize a list to store duplicates

We'll use a list to keep track of the numbers that appear more than once.

Step 2: Iterate through the array

We'll go through each element in the array and use another loop to check if this element appears again later in the array.

Step 3: Check for duplicates

If an element appears again later in the array, we'll add it to our duplicates list. We'll also make sure not to add it multiple times.

Step 4: Return the result

Finally, we'll return the list of duplicates.

Detailed Example in C#

C#

```
using System;
using System.Collections.Generic;

public class Solution
{
    public IList<int> FindDuplicates(int[] nums)
    {
        List<int> duplicates = new List<int>();

        // Iterate through the array
        for (int i = 0; i < nums.Length; i++)
        {
            // Check if the current element appears again in the array
            for (int j = i + 1; j < nums.Length; j++)
            {
                if (nums[i] == nums[j])
                {
                    // Check if the duplicate is already in the list
                    if (!duplicates.Contains(nums[i]))
                    {
                        duplicates.Add(nums[i]);
                    }
                    break;
                }
            }
        }

        return duplicates;
    }

    public static void Main(string[] args)
    {
        Solution solution = new Solution();
        int[] nums = { 4, 3, 2, 7, 8, 2, 3, 1 };
        IList<int> result = solution.FindDuplicates(nums);
        Console.WriteLine("Duplicates: " + string.Join(", ", result)); // Output:
Duplicates: 2, 3
    }
}
```

Explanation of the Code:

1. **Initialize a List for Duplicates**:
   List<int> duplicates = new List<int>();

   - We create a list to store the duplicate elements.
2. **Outer Loop to Iterate Through the Array**:
   for (int i = 0; i < nums.Length; i++)

   - We start a loop that goes through each element of the array.
3. **Inner Loop to Check for Duplicates**:
   for (int j = i + 1; j < nums.Length; j++)

   - For each element in the outer loop, we start another loop that checks the remaining elements in the array.
4. **Check if Elements are Equal**:
   if (nums[i] == nums[j])

   - We compare the current element from the outer loop with the current element from the inner loop.
5. **Add to Duplicates List if Not Already Present**:

```
C#

if (!duplicates.Contains(nums[i]))
6. {
7.     duplicates.Add(nums[i]);
8. }
```

    - If the elements are equal and the element is not already in the duplicates list, we add it.
9. **Return the Duplicates List**:
   return duplicates;

    - After processing all elements, we return the list of duplicates.

Complexity Analysis:

- **Time Complexity**: O(n^2), where `n` is the length of the array. This is because we use a nested loop to compare each element with every other element.
- **Space Complexity**: O(n), where `n` is the number of duplicate elements, because we store the duplicates in a list.

This approach is straightforward and easy to understand, making it a suitable choice for beginners. However, for large arrays, a more efficient approach (like the one using array indexing) would be preferable.

2. Use List and can use Contains

Complexity analysis -

- **Time Complexity**: O(n), where `n` is the length of the array. This is because we iterate through the array once, `List` operations (add and contains) are O(1) on average.
- **Space Complexity**: O(n), where `n` is the number of elements in the array. This is due to the additional space used by the `List`.